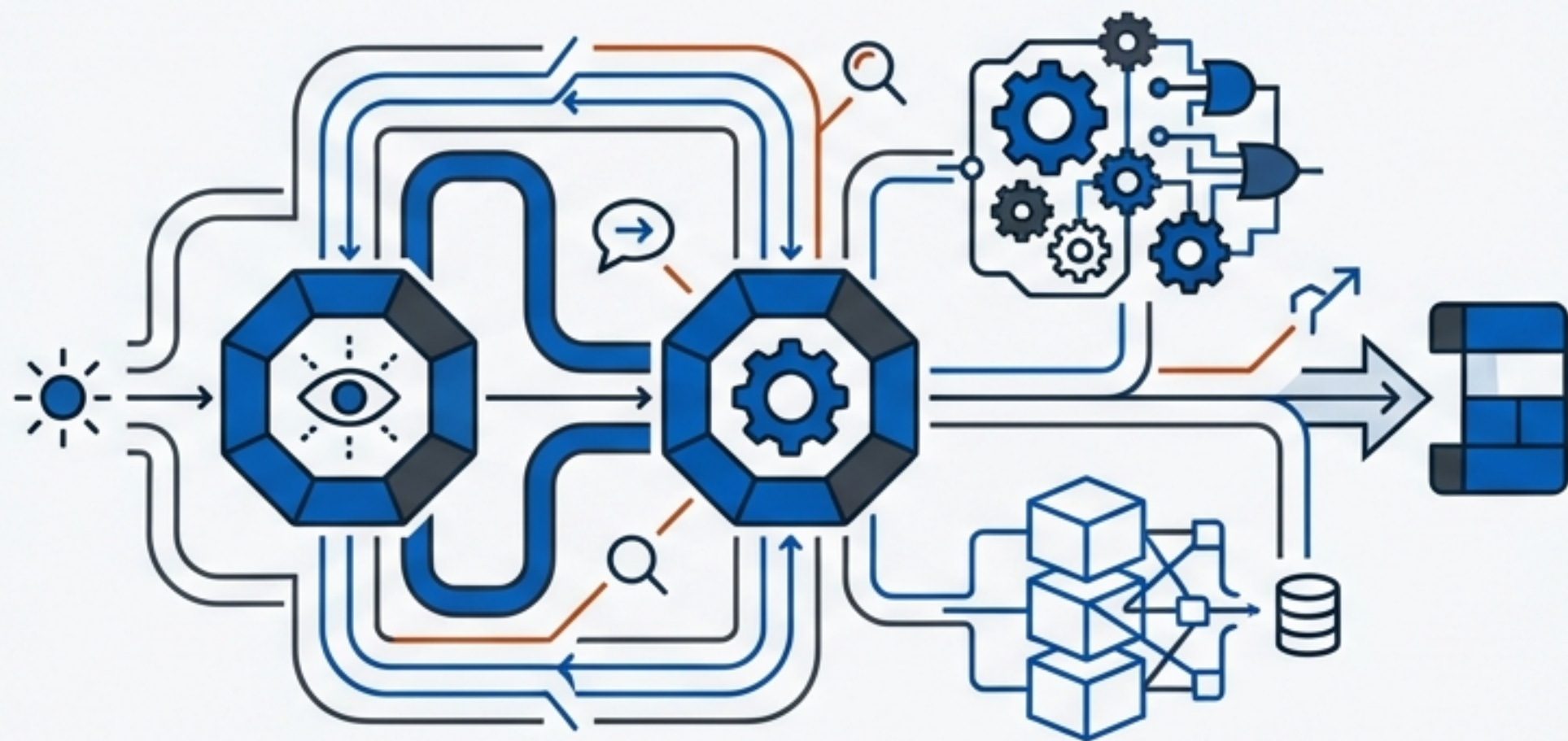


AI Agent 项目实战

从蓝图到杰作：一位构建者的旅程

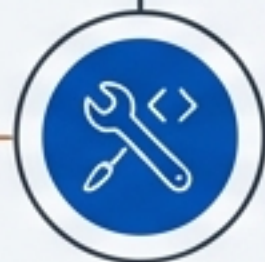
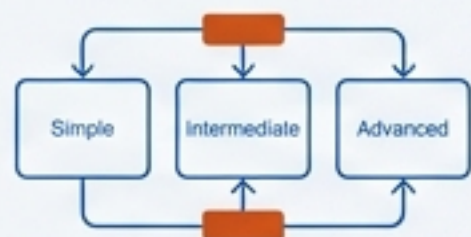
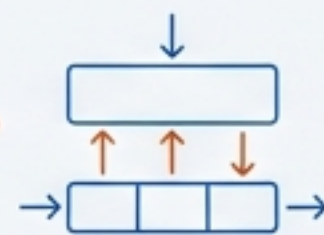


我们的构建者旅程

本次分享将引导你走过一条从理论到实践的完整路径。我们将从理解 Agent 的核心思维模式开始，逐步构建功能日益强大的智能体，最终完成一个可部署的全栈 AI 助理。

第一部分：奠定基石

探索 Agent 的核心理念与关键架构。

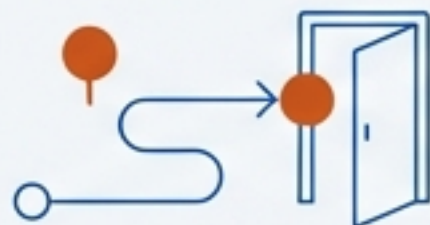
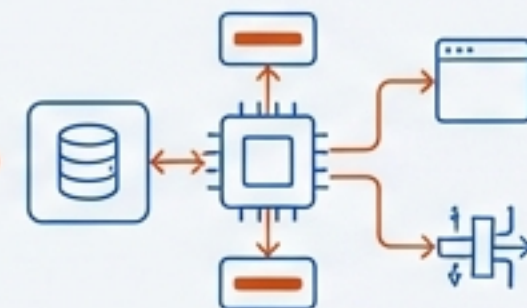


第二部分：实战工坊

由浅入深，动手构建三种不同能力的 Agent。

第三部分：终极项目

架构并实现一个多功能、有记忆的智能助理。



第四部分：总结展望

提炼核心模式，开启你的 Agent 构建之旅。

什么是 AI Agent? 从被动工具到主动思考者

AI Agent 的本质，是赋予大语言模型（LLM）自主规划、使用工具和解决复杂问题的能力。这是一种从“计算器”到“决策引擎”的范式转变。

“LLM 计算器”



被动响应，依赖输入内容。

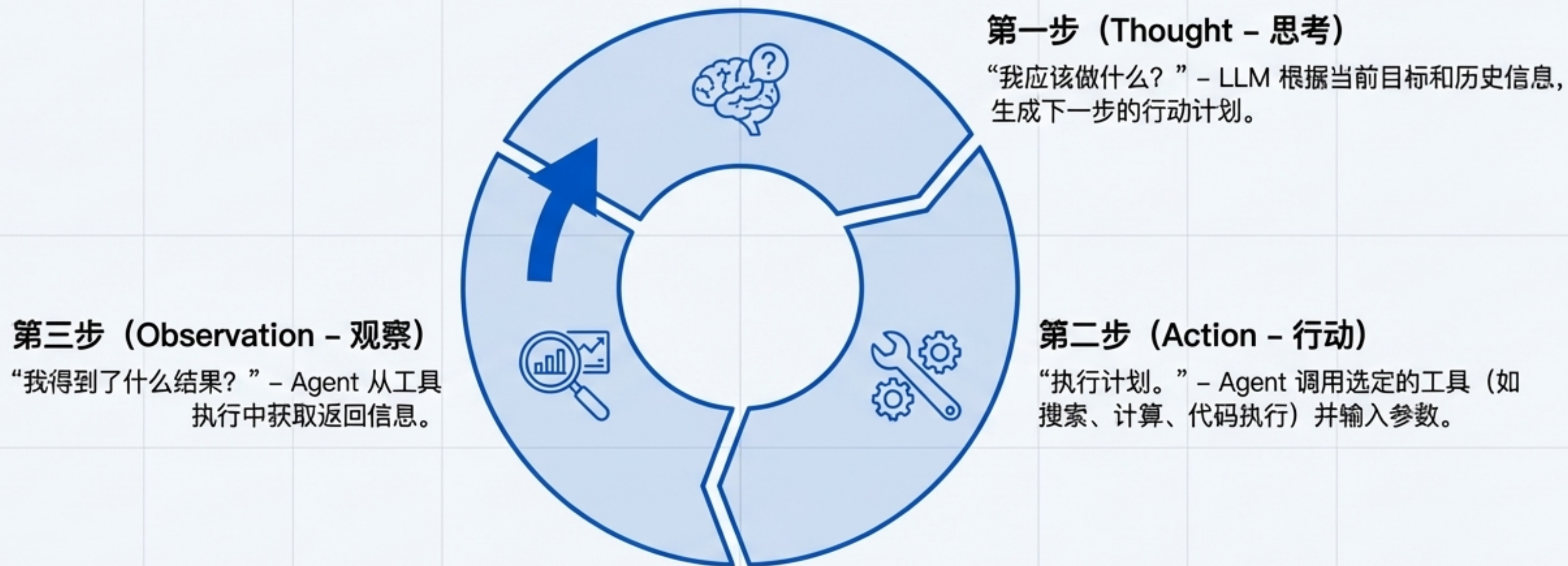
“Agent 决策引擎”



主动规划，循环执行，连接外部世界。

Agent 的心跳：ReAct 循环框架

ReAct (Reason + Act) 是 Agent 实现自主思考和行动的核心逻辑。它将复杂任务分解为一系列“思考 -> 行动 -> 观察”的迭代循环，直至问题解决。

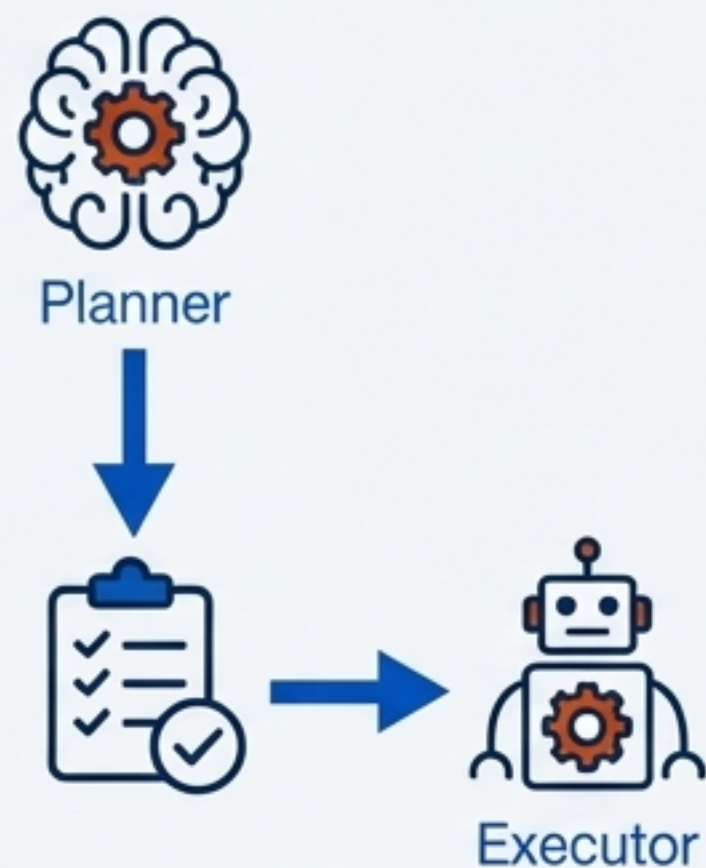


“`Thought: I need to search... Act: Search[...]`” 这就是 ReAct 在代码执行中的真实体现。

Agent 架构巡礼：解决问题的不同策略

针对不同类型的任务，社区已经发展出多种成熟的 Agent 架构。了解它们是选择正确工具的前提。

规划与执行 (Plan-and-Execute)



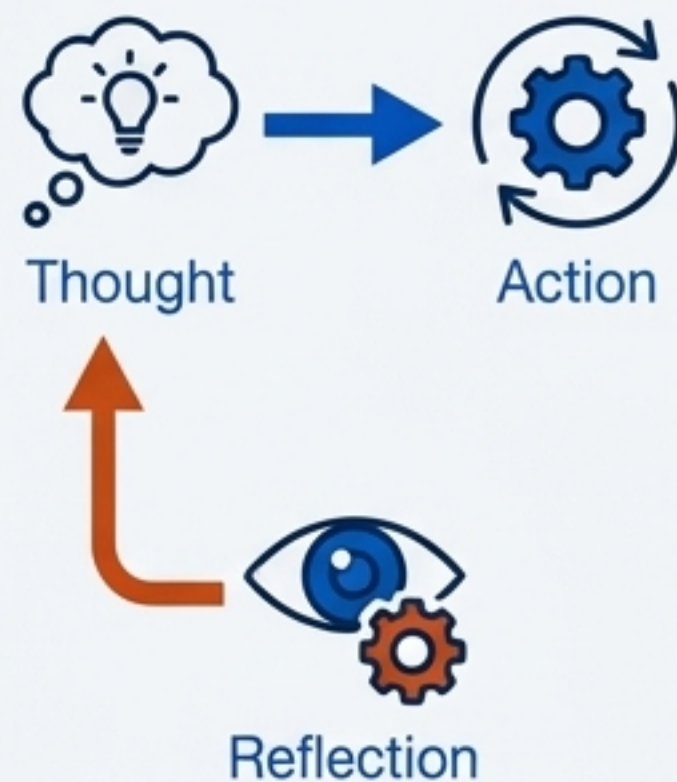
“先做计划，再执行”。适用于需要多个步骤、逻辑链条长的复杂任务。

自我提问 (Self-Ask)



将复杂问题分解为一系列简单的子问题，通过连续提问和寻找答案来解决。

思考与反思 (Thinking and Self-Reflection)



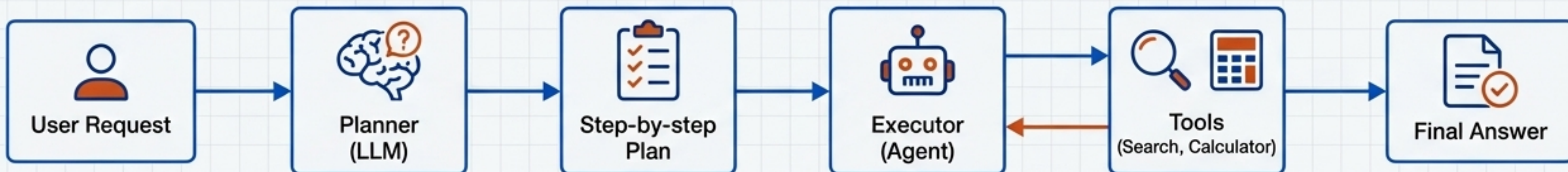
Agent 在执行后会进行自我评估和反思，从错误中学习并优化后续决策。

实战一：构建“规划与执行” Agent

目标 (Goal)

构建一个 Agent，使其能够为复杂问题（“在纽约，100美元能买几束玫瑰？”）制定一个清晰的计划，然后分步执行以得出答案。

蓝图 (The Blueprint)



砖块 (The Bricks) - 核心代码

左侧代码块 (定义工具)

```
# 导入库
from langchain_experimental.plan_and_execute import PlanAndExecute, load_agent_executor, load_chat_planner
from langchain.chains import LLMChain
from langchain_community.utilities import SerpAPIWrapper

# 定义工具
search = SerpAPIWrapper()
llm_math_chain = LLMChain.from_llm(llm, verbose=True)
tools = [
    Tool(name="Search", func=search.run, description="用于回答关于当前事件的问题"),
    Tool(name="Calculator", func=llm_math_chain.run, description="用于数学计算")
]
```

右侧代码块 (构建Agent)

```
# 加载规划器和执行器
planner = load_chat_planner(llm)
executor = load_agent_executor(llm, tools, verbose=True)

# 创建Plan and Execute代理
agent = PlanAndExecute(planner=planner, executor=executor, verbose=True)

# 运行
agent.run("在纽约，100美元能买几束玫瑰？")
```


观察执行：Agent 的思考与行动轨迹

第一步：生成计划 (Planner's Output)

```
> Entering new PlanAndExecute chain...
steps=[Step(value='查找当前纽约地区玫瑰花束的平均价格。'), Step(value='根据玫瑰花束的价格计算100美元能购买的花束数量。')]
```

Agent 首先将复杂问题分解为两个清晰、可执行的步骤。

第二步：执行步骤一 (Executor's Action)

```
> Entering new AgentExecutor chain...
Thought: To find the current average price of rose bouquets in the New York area, I will need to look up this information.
Action:
{
  "action": "Search",
  "action_input": "Current average price of rose bouquets in New York area"
}
```

执行器开始第一步，思考并决定使用“Search”工具。

第三步：观察与执行步骤二 (Observation & Next Action)

```
Observation: [... average price for a dozen roses is within $10 of the national average.... A bouquet of a dozen roses costs just $68.33 on average.]
Thought: I now have the average price. I need to calculate how many bouquets can be bought for $100.
Action:
{
  "action": "Calculator",
  "action_input": "100 / 68.33"
}
```

获得搜索结果后，Agent 决定使用“Calculator”工具完成第二步。

第四步：得出最终答案 (Final Answer)

```
Response: With $100, you can purchase 1 bouquet of roses.
```

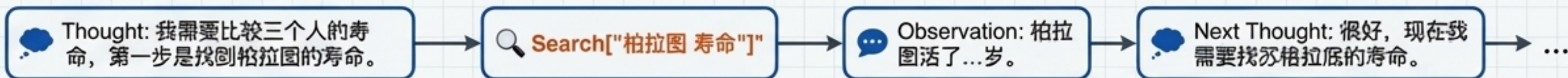
Agent 综合所有步骤的结果，给出了最终答案。

实战二：构建具备深度思考能力的 ReAct Agent

目标 (Goal)

构建一个能够“自言自语”的 Agent，通过分解问题、搜索信息并综合，来回答需要多步推理的事实性问题，例如：“柏拉图、苏格拉底和亚里士多德谁更长寿？”

蓝图 (The Blueprint)



砖块 (The Bricks) - 核心代码

```
from langchain.agents import load_tools, initialize_agent, AgentType
from langchain_openai import OpenAI

# 初始化LLM和工具
llm = OpenAI(temperature=0)
tools = load_tools(["serpapi"], llm=llm)

# 初始化Agent
# ZERO_SHOT_REACT_DESCRIPTION 是最经典、最通用的ReAct实现之一
agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)

# 运行
agent.run("Who lived longer: Plato, Socrates, or Aristotle?")
```

与 Plan-and-Execute 不同，ReAct Agent 的实现非常简洁。核心在于选择正确的 `AgentType`，框架会处理底层的循环逻辑。

深入机器之心：追踪 ReAct 的完整推理链

Who lived longer: Plato, Socrates, or Aristotle?

第一轮循环：寻找柏拉图的寿命

****Thought****: I should search for the lifespans of Plato, Socrates, and Aristotle to compare.

****Action****: Search | ****Action Input****: Plato lifespan

****Observation****: ...Plato, the Athenian philosopher, was born in 428–7 B.C.E and died at the age of eighty or eighty-one...

第二轮循环：寻找苏格拉底的寿命

****Thought****: I need to find the lifespan of Socrates next.

****Action****: Search | ****Action Input****: Socrates lifespan

****Observation****: Socrates (born c. 470 bce, Athens [Greece]–died 399 bce, Athens) was an ancient Greek philosopher...

第三轮循环：寻找亚里士多德的寿命

****Thought****: I should now search for the lifespan of Aristotle to compare with Plato and Socrates.

****Action****: Search | ****Action Input****: Aristotle lifespan

****Observation****: Aristotle's lifespan (384 – 322)... Aristotle died at age 62...

最终思考与答案

****Final Thought****: I now know the final answer.

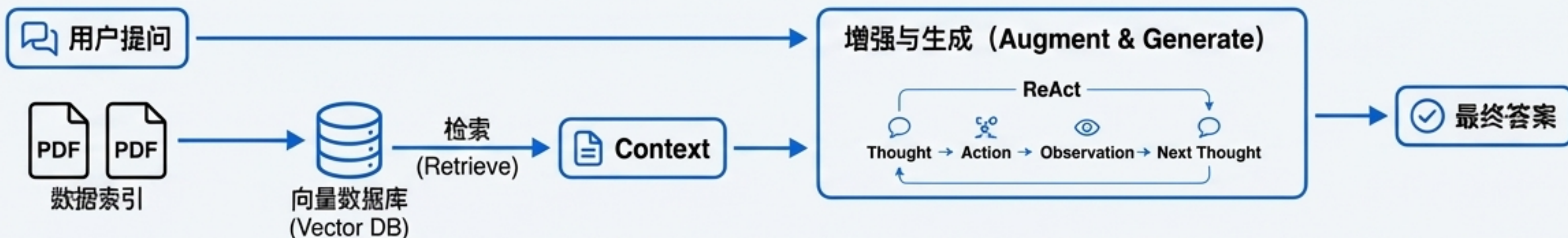
****Final Answer****: Plato lived the longest, dying at the age of 80. Socrates died at 71, and Aristotle died at 62.

实战三：构建知识库增强（RAG）Agent

目标（Goal）

构建一个能回答特定领域问题的 Agent，其知识来源于我们提供的私有文档（例如，两家公司的PDF格式财报），而非其通用训练数据。

蓝图（The Blueprint）：RAG + ReAct



框架聚焦：LlamaIndex

为什么选择它？LlamaIndex 是一个以数据为中心的框架，专为构建基于私有数据的 LLM 应用而设计，极大地简化了数据加载、索引和查询的流程。

砖块（The Bricks） - 数据摄入与索引

```
from llama_index.core import SimpleDirectoryReader, VectorStoreIndex, StorageContext

# 1. 加载PDF文档
A_docs = SimpleDirectoryReader(input_files=["./A.pdf"]).load_data()
B_docs = SimpleDirectoryReader(input_files=["./B.pdf"]).load_data()

# 2. 从文档创建向量索引
A_index = VectorStoreIndex.from_documents(A_docs)
B_index = VectorStoreIndex.from_documents(B_docs)

# 3. (可选) 持久化索引到本地存储
A_index.storage_context.persist(persist_dir="./storage/A")
B_index.storage_context.persist(persist_dir="./storage/B")
```


RAG Agent 实战：封装知识库为工具并执行

砖块（The Bricks） - 创建工具与 Agent

我们需要将每个文档的索引（Index）包装成 Agent 可以调用的工具（Tool）。这样，Agent 就能像使用“搜索”或“计算器”一样，使用我们的私有知识库。

左侧代码块（创建查询引擎工具）

```
from llama_index.core.tools import QueryEngineTool, ToolMetadata

# 将索引转换为查询引擎
A_engine = A_index.as_query_engine(similarity_top_k=3)
B_engine = B_index.as_query_engine(similarity_top_k=3)

# 封装为工具，提供清晰的名称和描述
query_engine_tools = [
    QueryEngineTool(
        query_engine=A_engine,
        metadata=ToolMetadata(
            name="A_Finance",
            description="用于提供A公司的财务信息",
        ),
    ),
    QueryEngineTool(
        query_engine=B_engine,
        metadata=ToolMetadata(
            name="B_Finance",
            description="用于提供B公司的财务信息",
        ),
    ),
]
```

右侧代码块（创建并运行Agent）

```
from llama_index.core.agent import ReActAgent

# 从工具列表创建ReAct Agent
agent = ReActAgent.from_tools(
    query_engine_tools,
    llm=llm,
    verbose=True
)

# 运行
response = agent.chat("Compare the sales of the two companies")
print(response)
```

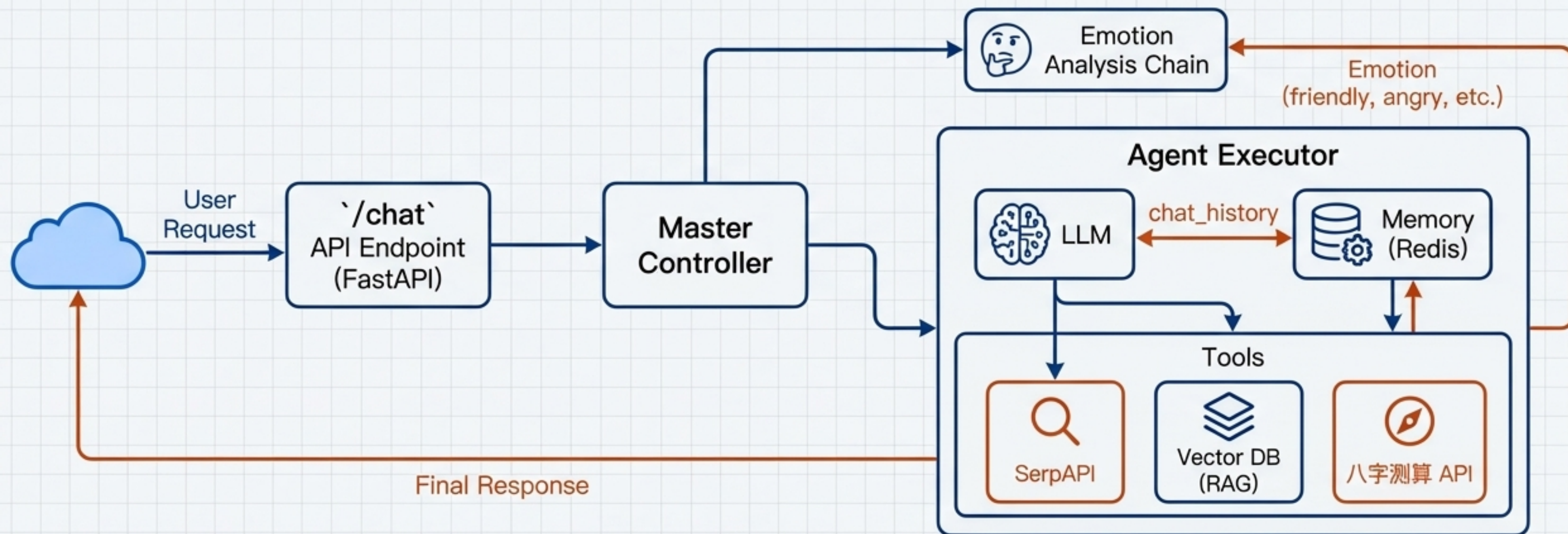
构建（The Building） - 智能决策

当 Agent 运行时，它会分析问题，并根据工具的 `description`（“用于提供A公司的财务信息”）来智能地决定是调用 `A\_Finance` 工具还是 `B\_Finance` 工具，或者两者都调用，以找到最准确的答案。

综合项目：构建多功能、有记忆的智能助理

终极挑战 (The Grand Challenge)

架构一个完整的、由 API 驱动的对话式 Agent。它不仅要能调用多种工具，还要具备独特的“人设”、能感知用户情绪，并通过 Redis 拥有长期记忆。



这是一个典型的生产级 Agent 系统，将 Agent Core 与 Web 服务、数据库和外部 API 深度集成。

特性深度解析：赋予 Agent 人格、情感与定制能力

特性一：人格设定 (Persona)

通过一个精心设计的系统提示词 (System Prompt)，我们为 Agent 注入了独特的角色和个性。

```
# 设置系统角色定位
self.SYSTEM = """你是一个非常资深的算命先生。你叫陈玉楼，人称陈大师。
以下是你的个人设定：
1. 你精通阴阳五行，能够算命、紫薇斗数、姓名测算...
2. 你大约60岁左右，过去曾是湘西一带赫赫有名的土匪头子...
...
"""
```

特性二：情绪感知 (Emotion)

在主 Agent 运行前，我们用一个独立的 prompt chain 来判断用户情绪，以便 Agent 能做出更具同理心的回应。

```
prompt = """根据用户的输入判断用户的情绪，
回应的规则如下：
1. 如果用户输入的内容偏向于负面情绪，只返回"depressed"，不要...
2. 如果用户输入的内容偏向于正面情绪，只返回"friendly"，不要...
...
用户的输入内容是：{query}"""
chain = ChatPromptTemplate.from_template(prompt) | ChatOpenAI(...)
```

特性三：自定义工具 (Custom Tools)

Agent 的强大之处在于其扩展性。我们可以轻松编写函数，通过 **@tool** 装饰器将其封装成 Agent 可用的工具。

```
from langchain.tools import tool

@tool
def bazi_cesuan(query: str):
    """只有做八字排盘的时候才会使用这个工具，
    需要输入用户名和出生年月日时。如果缺少...
    url = f"https://api.yuanfenju.com/index.php/v1/Bazi/cesuan"
    # ... (API call logic)
    return result
```


上线服务：使用 FastAPI 封装 Agent 并提供 API

仅有强大的 Agent 逻辑是不够的，我们还需要将其工程化，通过标准的 Web API 对外提供服务。FastAPI 是 Python 生态中实现这一目标的理想选择。

砖块（The Bricks）– 核心服务代码

server.py

```
import uvicorn
from fastapi import FastAPI

# 导入我们定义的主控类
from master import Master

app = FastAPI()

# 定义核心聊天API端点
@app.post("/chat")
def chat(query: str):
    # 实例化主控类
    master = Master()
    # 运行主逻辑并返回结果
    return master.run(query)

# 启动服务
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

项目展示（Project Showcase）

FastAPI

default

GET

/

POST

/chat

POST

/add_urls

FastAPI 自动为我们的代码生成了交互式 API 文档，极大地方便了测试和集成。

POST /chat

Parameters

Name	Description
query (query)	你是谁

Execute

Responses

Err response icon JSON mode:

```
{
  "response": "我是你的智能助手，陈久叙。有什么可以帮你的吗?"
}
```

任何人现在都可以通过这个简单的 API 接口，与我们构建的复杂 Agent 系统进行交互。

构建者手册：核心要点回顾与你的下一步

三大核心架构模式 (Architectural Patterns)



规划与执行：适用于需要严谨步骤的复杂任务。



ReAct：动态、通用的实时推理框架，是大多数 Agent 的基础。



RAG：连接私有数据与 LLM 的桥梁，让 Agent 成为真正的领域专家。

四大基本组件 (Essential Components)

- **LLM (大脑)：** Agent 的推理和语言核心。
- **工具 (Tools)：** Agent 连接物理和数字世界的“双手”。
- **记忆 (Memory)：** 赋予 Agent 上下文感知和持续学习的能力。
- **执行器 (Executor)：** 驱动一切、协调所有组件的“引擎”。

你的旅程从此开始：你已经掌握了构建 AI Agent 的核心蓝图与关键技术。从简单的工具调用到复杂的全栈应用，这些模式是你创造下一代智能应用的基石。现在，开始构建你自己的 Agent 吧。